

Федеральное государственное автономное образовательное учреждение высшего
образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –

Системное и прикладное программное обеспечение

Отчёт

По лабораторной работе №1

Проектирование вычислительных систем

Вариант: 3

Выполнили:

студенты 3 курса

Батманов Даниил Евгеньевич

Разинкин Александр Владимирович

Группа: Р3307

Принял:

Пинкевич Василий Юрьевич

Отчёт принят «__»____ 2025 г.

Оценка: _____

г. Санкт-Петербург, 2025

Цели работы

1. Получить базовые знания о принципах устройства стенда SDK-1.1M и программировании микроконтроллеров.
2. Изучить устройство интерфейсов ввода-вывода общего назначения (GPIO) в микроконтроллерах и приемы использования данных интерфейсов.

Задание

Разработать и реализовать драйверы управления светодиодными индикаторами и чтения состояния кнопки стенда SDK-1.1M (расположены на боковой панели стенда).

Обработка нажатия кнопки в программе должна включать программную защиту от дребезга.

Функции и другие компоненты драйверов должны быть универсальными, т. е. пригодными

для использования в любом из вариантов задания и не должны содержать прикладной логики программы. Функции драйверов должны быть неблокирующими, то есть не должны содержать ожиданий события (например, нажатия кнопки). Также, в драйверах не должно быть пауз с активным ожиданием (функция `HAL_Delay()` и собственные варианты с аналогичной функциональностью).

При необходимости параллельного выполнения разных процессов (например, опроса кнопки и ожидания перед переключением светодиодов) следует организовать псевдопараллельную работу процессов в стиле кооперативной многозадачности. Это возможно сделать без существенной потери точности соблюдения необходимых интервалов времени между действиями, поскольку действия выполняются очень быстро по сравнению с промежутками ожидания между ними. Каждый процесс (который может быть выражен всего в нескольких строках кода) должен использовать не активное ожидание (`HAL_Delay()`), а считывать текущее значение миллисекундного счетчика (`HAL_GetTick()`), проверяя, прошло ли необходимое количество времени для выполнения следующего действия. После проверки и (при необходимости) выполнения действия управление должно передаваться следующему процессу, чтобы он тоже мог провести такую проверку.

Контакты подключения кнопки и светодиодов должны быть настроены в режиме GPIO.

Использование прерываний и дополнительных таймеров (кроме `SysTick`) не допускается.

Написать программу с использованием разработанных драйверов в соответствии с вариантом задания.

Порядок выполнения работы

1. Изучить:

– устройство и инструкцию по эксплуатации стенда SDK-1.1M;

- основные характеристики микроконтроллера STM32 по листу данных (datasheet); документация выложена на сайте производителя [34] (например, см. документацию микроконтроллера STM32F427VIT6 [41], обозначение документа – DS9405 [7]);
 - разделы учебного пособия:
 - о 1.1. Сигналы сброса и синхронизации;
 - о 1.2. Интерфейс ввода-вывода общего назначения (GPIO);
 - электрическую принципиальную схему стенда в части управления светодиодами и кнопкой, используемыми в данной работе;
 - раздел «General purpose I/O (GPIO)» из справочного руководства (reference manual) по микроконтроллерам линейки STM32F4 (обозначение документа – RM0090 [4]), знать устройство портов ввода-вывода, режимы их работы и способы настройки.
2. Создать и настроить пустой проект программы для SDK-1.1M.
 3. Настроить тактовые частоты.
 4. Настроить сигналы GPIO, необходимые для выполнения задания.
 5. Изучить:
 - состав стандартного драйвера GPIO из библиотеки HAL (файлы `stm32f4xx_hal_gpio.c/.h` в проекте), знать основные определения и функции, принцип работы драйвера. Справочная информация находится в комментариях в файле драйвера в документации (обозначение документа – UM1725 Description of STM32F4 HAL and low-layer drivers [42]) с сайта производителя;
 - содержимое инициализационного кода в созданных генератором файлах `gpio.c/.h`.
 6. Разработать драйверы управления светодиодами и чтения состояния кнопки.
 7. Разработать программу согласно варианту задания и провести ее тестирование.
 8. Модифицировать программу: отключить в графическом конфигураторе настройку портов GPIO. Разработать собственные функции для инициализации и использования портов GPIO (можно с использованием объявленных в стандартной библиотеке структур и констант). Заменить в программе использование библиотечных функций работы с GPIO на вызов собственных реализаций.

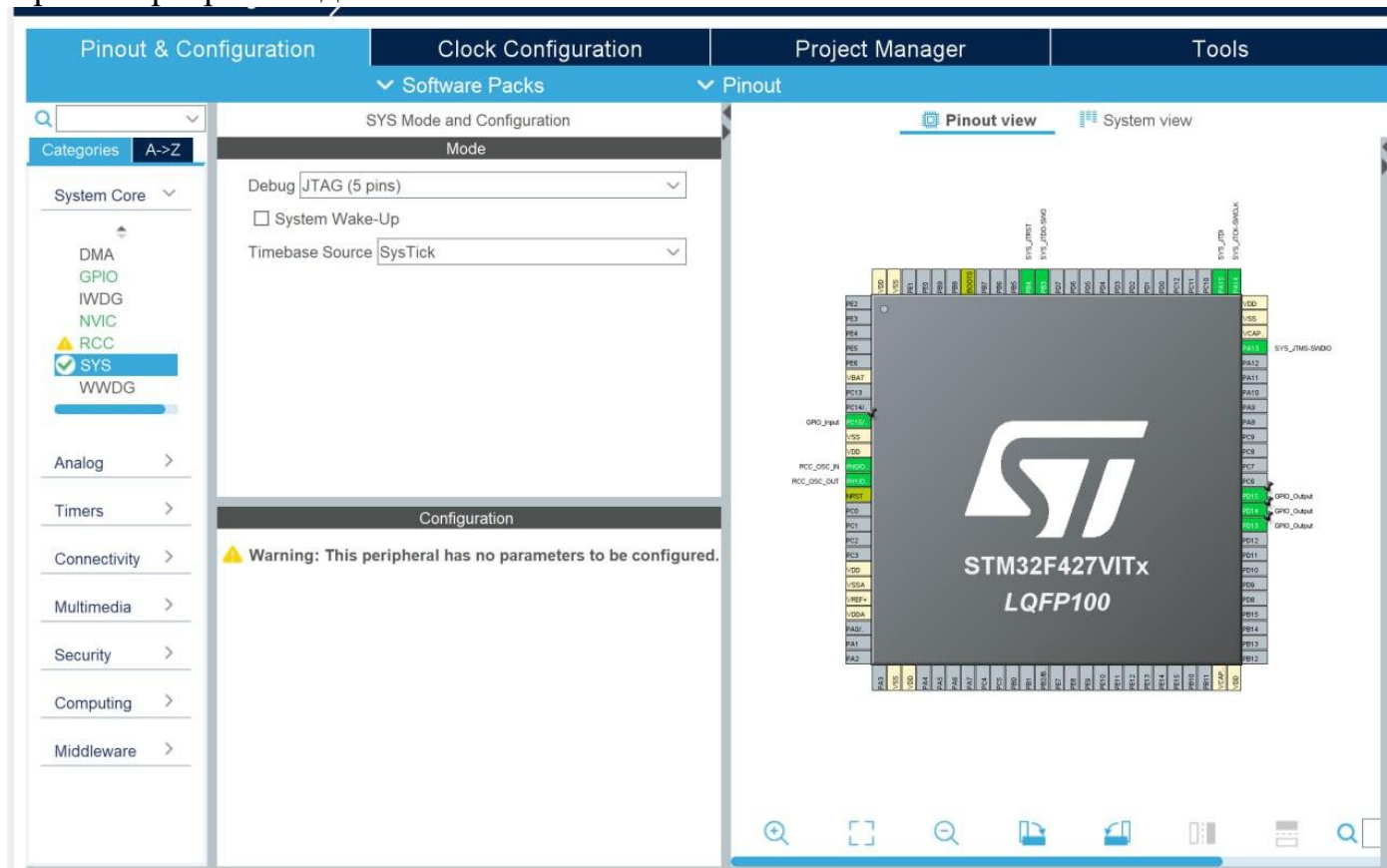
Вариант

Реализовать «передатчик» азбуки Морзе. Программа должна запоминать последовательность нажатий кнопки (короткое – точка, длинное – тире) длиной до 8

нажатий. При этом после каждого нажатия кнопки двухцветный светодиод должен коротким мерцанием показывать, какой сигнал был введён: точка – желтым, тире – красным. После окончания ввода (долгая пауза после последнего нажатия кнопки) последовательность один раз «отправляется» при помощи зелёного светодиода быстрыми (точка) и долгими (тире) импульсами. После этого программа снова переходит в режим ввода последовательности.

Ход работы

Проект программы для SDK-1.1M:



Настройки тактовой частоты:

Categories

A->Z

System Core

DMA

GPIO

IWDG

NVIC

RCC

SYS

WWDG

Analog

RCC Mode and Configuration

Mode

High Speed Clock (HSE)

Crystal/Ceramic Resonator

Low Speed Clock (LSE)

Disable

☐

Master Clock Output 1

☐

Master Clock Output 2

☐

Audio Clock Input (I2S_CKIN)

lab1.ioc - Clock Configuration

Pinout & Configuration

Clock Configuration

Project Manager

Tools

Resolve Clock Issues

The diagram illustrates the clock configuration for a microcontroller, showing the flow from input frequencies to various system and peripheral clocks.

Input Frequencies:

- 32.768 KHz (LSE)
- 0-1000 KHz (LSI RC)
- 16 MHz (HSI RC)
- 4-26 MHz (HSE)
- 12.288 MHz (MCO2 source Mux)

Key Components and Dividers:

- RTC Clock Mux:** Selects between HSE (/2), LSE, and LSI.
- System Clock Mux:** Selects between HSI, HSE, and PLLCLK.
- Main PLL:** Multiplier X 216, Divider /2 (to SYSCLK), and Divider /4 (to I/Q).
- PLL12S:** Multiplier X 192, Divider /2 (to PLL12SQCLK), and Divider /2 (to I/Q).
- AHB Prescaler:** /1
- APB1 Prescaler:** /4
- APB2 Prescaler:** /2
- PLL2SAICLK:** /1

Outputs and Clocks:

- 32 KHz:** To RTC (KHz), To IWDG (KHz)
- 180 MHz:** SYSCLK (MHz), HCLK (MHz) (180 MHz max)
- 180 MHz:** Ethernet PTP clock (MHz), HCLK to AHB bus, core, memory and DMA (MHz), To Cortex System timer (MHz), FCLK Cortex clock (MHz)
- 45 MHz max:** PCLK1 (MHz)
- 90 MHz max:** PCLK2 (MHz)
- 45 MHz:** APB1 peripheral clocks (MHz)
- 90 MHz:** APB1 Timer clocks (MHz)
- 90 MHz:** APB2 peripheral clocks (MHz)
- 180 MHz:** APB2 timer clocks (MHz)
- 90 MHz:** 48MHz clocks (MHz)
- 150 MHz:** I2S clocks (MHz)
- 20.416667 MHz:** SAI1-A clocks (MHz)

Other Labels:

- Enable CSS
- I2S source Mux
- SAI1-A source Mux
- PLL2SAICLK
- PLL12SQCLK
- PLL12S
- PLL Source Mux
- HSE
- LSI
- LSE
- HSI
- RTC
- IWDG
- APB1
- APB2
- AHB
- SAI1-A
- I2S
- SAI1-A

Программа:

```
/* USER CODE BEGIN Header */
/**
```

```

*****
****
* @file      : main.c
* @brief     : Main program body

*****
****
* @attention
*
* Copyright (c) 2025 STMicroelectronics.
* All rights reserved.
*
* This software is licensed under terms that can be found in the LICENSE file
* in the root directory of this software component.
* If no LICENSE file comes with this software, it is provided AS-IS.
*

*****
****
*/
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"

/* Private includes -----*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */
/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/

```

```

/* USER CODE BEGIN PV */
uint8_t signals = 0;
size_t signals_count = 8;
int index = 7;
/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
/* USER CODE BEGIN PFP */
void sparkIndicator(uint16_t GPIO_Pin, uint32_t delay, int times);
/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */
/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    /* USER CODE BEGIN 2 */

```

```

uint32_t wait = 500;
uint32_t debounce_delay = 50; // 50 мс для антидребезга
uint32_t last_press_time = 0;
_Bool button_valid = 0;

/* USER CODE END 2 */
/* Infinite loop */
/* USER CODE BEGIN WHILE */

while(1) {
    int button_state = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_15);

    if (button_state == 0) { // кнопка нажата (низкий уровень)
        if (!button_valid) {
            if (HAL_GetTick() - last_press_time > debounce_delay) {
                button_valid = 1;
                last_press_time = HAL_GetTick();

                // Обработка нажатия кнопки:
                int is_long_press = 0;
                uint32_t start_time = HAL_GetTick();

                while (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_15) == 0){
                    if (HAL_GetTick() - start_time >= wait) {
                        is_long_press = 1;
                        break;
                    }
                }

                if (is_long_press) {
                    sparkIndicator(GPIO_PIN_15, 100, 5);
                    signals |= (1 << index); // установка бита
                } else {
                    sparkIndicator(GPIO_PIN_14, 100, 5);
                    signals &= ~(1 << index); // сброс бита
                }
                index--;
                if (index < 0) {
                    // Выводим сигнал по очереди с задержками
                    for (int i = 0; i < signals_count; i++) {
                        if ((signals & (1 << (signals_count - 1 - i))) == 0) {
                            sparkIndicator(GPIO_PIN_13, 250, 1);
                        } else {
                            sparkIndicator(GPIO_PIN_13, 1000, 1);
                        }
                    }
                }
            }
        }
    }
}

```



```

        index = signals_count - 1;
        signals = 0;
    }
}
} else {
    button_valid = 0; // кнопка отпущена, сбрасываем флаг
}
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage
     */
    __HAL_RCC_PWR_CLK_ENABLE();

    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = 15;
    RCC_OscInitStruct.PLL.PLLN = 216;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
    RCC_OscInitStruct.PLL.PLLQ = 4;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
}

```

```

/** Activate the Over-Drive mode
*/
if (HAL_PWREx_EnableOverDrive() != HAL_OK)
{
    Error_Handler();
}

/** Initializes the CPU, AHB and APB buses clocks
*/
RCC_ClkInitStruct.ClockType                                =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                    |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5) != HAL_OK)
{
    Error_Handler();
}
}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOD_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOD,                                GPIO_PIN_13|GPIO_PIN_14|GPIO_PIN_15,
GPIO_PIN_RESET);

    /*Configure GPIO pin : PC15 */
    GPIO_InitStruct.Pin = GPIO_PIN_15;

```

```

GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

/*Configure GPIO pins : PD13 PD14 PD15 */
GPIO_InitStruct.Pin = GPIO_PIN_13|GPIO_PIN_14|GPIO_PIN_15;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);

}

/* USER CODE BEGIN 4 */
void sparkIndicator(uint16_t GPIO_Pin, uint32_t delay, int times)
{
    for (int i = 0; i < times; i++) {
        HAL_GPIO_WritePin(GPIOD, GPIO_Pin, GPIO_PIN_SET);
        HAL_Delay(delay);
        HAL_GPIO_WritePin(GPIOD, GPIO_Pin, GPIO_PIN_RESET);
        HAL_Delay(100);
    }
}
/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number

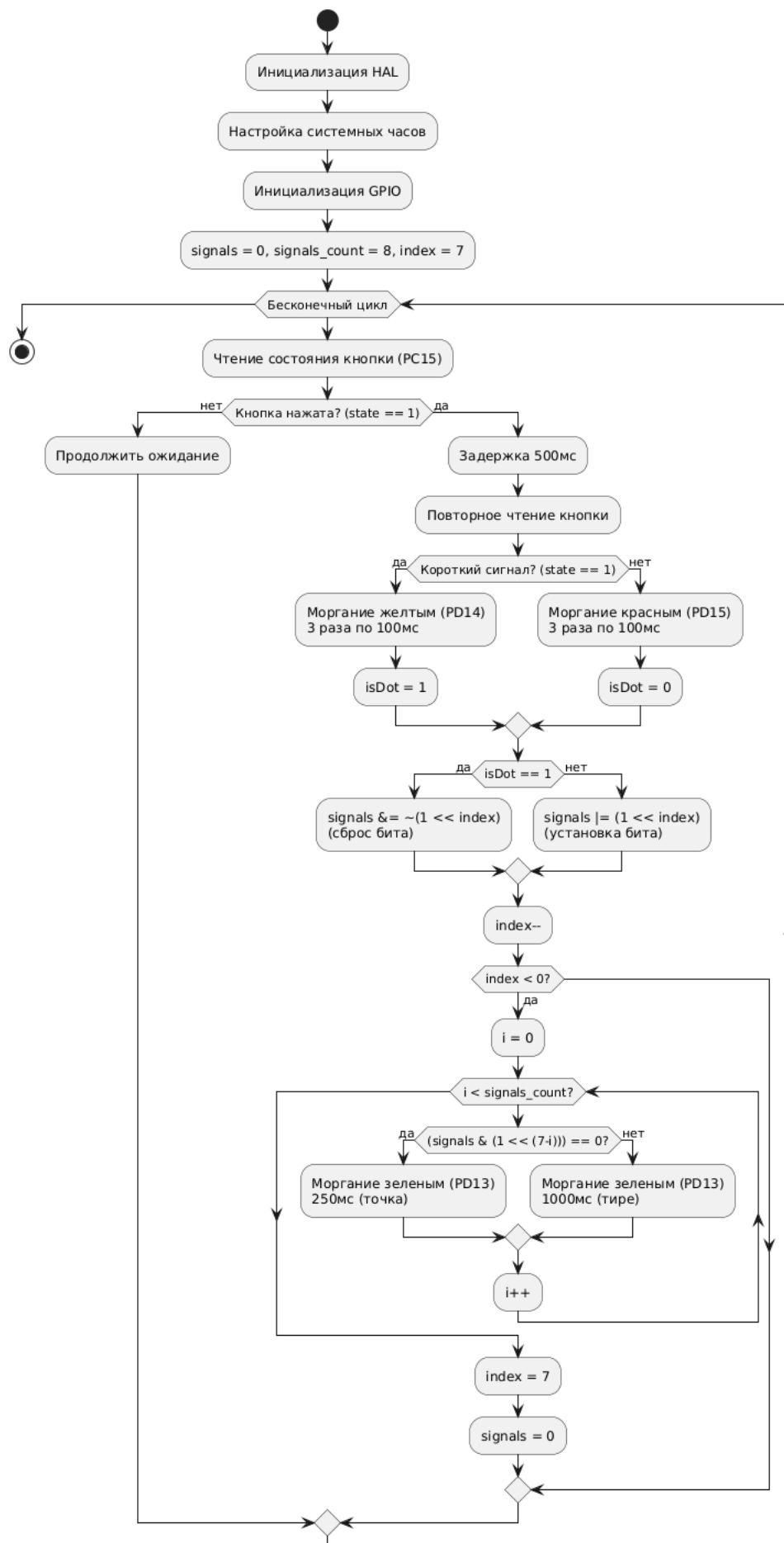
```

```

* @retval None
*/
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
       ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```

Блок-схема:



Заключение

Положено начало освоения GPIO. Разработаны неблокирующие драйверы для светодиодов и кнопки стенда SDK-1.1M. Реализован передатчик азбуки Морзе с обработкой 8 сигналов, индикацией точек/тире и выводом последовательности. Драйверы универсальны и соответствуют требованиям задания.